



```
In [2]: import numpy as np
import pandas as pd
```

```
In [ ]:
```

```
In [3]: df = pd.read_csv('spam.csv')
```

```
In [4]: df.sample(5)
```

```
Out[4]:
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
5365	spam	Camera - You are awarded a SiPix Digital Camer...	NaN	NaN	NaN
2154	ham	Oh god i am happy to see your message after 3 ...	NaN	NaN	NaN
3409	ham	Joy's father is John. Then John is the ____ of...	NaN	NaN	NaN
1779	spam	Loan for any purpose 💎500 - 💎75,000. Homeowner...	NaN	NaN	NaN
3320	ham	Yo im right by yo work	NaN	NaN	NaN

```
In [5]: df.shape
```

```
Out[5]: (5572, 5)
```

```
In [ ]: # 1. Data cleaning
# 2. EDA
# 3. Text Preprocessing
# 4. Model building
# 5. Evaluation
# 6. Improvement
# 7. Website
# 8. Deploy
```

```
In [6]: ##1.Data Cleaning
```

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   v1           5572 non-null   object
1   v2           5572 non-null   object
2   Unnamed: 2   50 non-null     object
3   Unnamed: 3   12 non-null     object
4   Unnamed: 4   6 non-null      object
dtypes: object(5)
memory usage: 217.8+ KB
```

```
In [8]: # drop last 3 cols
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], inplace=True)
```

```
In [9]: df.sample(5)
```

```
Out[9]:
```

	v1	v2
4000	ham	He's just gonna worry for nothing. And he won't...
2216	ham	OK i'm waliking ard now... Do u wan me 2 buy a...
3278	ham	Solve d Case : A Man Was Found Murdered On &l...
4228	ham	Have you bookedthe hut? And also your time off...
199	ham	Found it, ENC <#> , where you at?

```
In [10]: # renaming the cols
df.rename(columns={'v1': 'target', 'v2': 'text'}, inplace=True)
df.sample(5)
```

```
Out[10]:
```

	target	text
2274	ham	Think I could stop by in like an hour or so? M...
4640	ham	Havent.
3250	ham	I??Il leave around four, ok?
3599	ham	I know you mood off today
2544	ham	She is our sister.. She belongs 2 our family....

```
In [11]: from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
```

```
In [12]: df['target'] = encoder.fit_transform(df['target'])
```

```
In [13]: df.head()
```

```
Out[13]:
```

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

```
In [14]: # missing values
df.isnull().sum()
```

```
Out[14]: target    0
text          0
dtype: int64
```

```
In [15]: # check for duplicate values
df.duplicated().sum()
```

```
Out[15]: np.int64(403)
```

```
In [16]: # remove duplicates
df = df.drop_duplicates(keep='first')
```

```
In [17]: df.duplicated().sum()
```

```
Out[17]: np.int64(0)
```

```
In [18]: df.shape
```

```
Out[18]: (5169, 2)
```

```
In [ ]: # 2.EDA
```

```
In [19]: df.head()
```

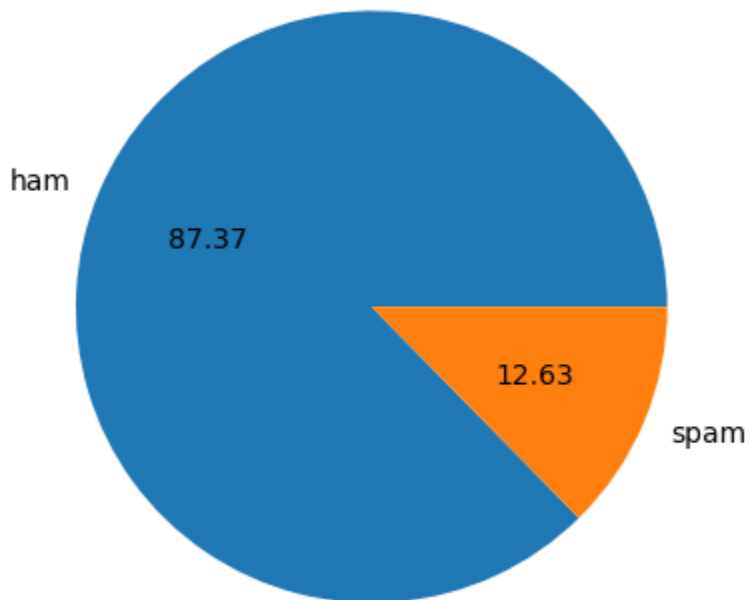
```
Out[19]:
```

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

```
In [20]: df['target'].value_counts()
```

```
Out[20]: target
0      4516
1       653
Name: count, dtype: int64
```

```
In [21]: import matplotlib.pyplot as plt
plt.pie(df['target'].value_counts(), labels=['ham', 'spam'], autopct="%0.2f")
plt.show()
```



```
In [ ]: # Data is imbalanced
```

```
In [22]: import nltk
```

```
In [23]: !pip install nltk
```

```
Requirement already satisfied: nltk in c:\users\monde\anaconda3\lib\site-packages (3.9.1)
Requirement already satisfied: click in c:\users\monde\anaconda3\lib\site-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in c:\users\monde\anaconda3\lib\site-packages (from nltk) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in c:\users\monde\anaconda3\lib\site-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in c:\users\monde\anaconda3\lib\site-packages (from nltk) (4.67.1)
Requirement already satisfied: colorama in c:\users\monde\appdata\roaming\python\python313\site-packages (from click->nltk) (0.4.6)
```

```
In [24]: nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\monde\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

Out[24]: True

```
In [25]: df['num_characters'] = df['text'].apply(len)
```

```
In [26]: df.head()
```

```
Out[26]:
```

	target	text	num_characters
0	0	Go until jurong point, crazy.. Available only ...	111
1	0	Ok lar... Joking wif u oni...	29
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155
3	0	U dun say so early hor... U c already then say...	49
4	0	Nah I don't think he goes to usf, he lives aro...	61

```
In [27]: # num of words
df['num_words'] = df['text'].apply(lambda x:len(nltk.word_tokenize(x)))
```

```
In [28]: df.head()
```

```
Out[28]:
```

	target	text	num_characters	num_words
0	0	Go until jurong point, crazy.. Available only ...	111	24
1	0	Ok lar... Joking wif u oni...	29	8
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37
3	0	U dun say so early hor... U c already then say...	49	13
4	0	Nah I don't think he goes to usf, he lives aro...	61	15

```
In [29]: df['num_sentences'] = df['text'].apply(lambda x:len(nltk.sent_tokenize(x)))
```

```
In [30]: df.head()
```

Out[30]:

	target	text	num_characters	num_words	num_sentences
0	0	Go until jurong point, crazy.. Available only ...	111	24	2
1	0	Ok lar... Joking wif u oni...	29	8	2
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2
3	0	U dun say so early hor... U c already then say...	49	13	1
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1

In [31]: `df[['num_characters', 'num_words', 'num_sentences']].describe()`

Out[31]:

	num_characters	num_words	num_sentences
count	5169.000000	5169.000000	5169.000000
mean	78.923776	18.456761	1.966531
std	58.174846	13.325633	1.449833
min	2.000000	1.000000	1.000000
25%	36.000000	9.000000	1.000000
50%	60.000000	15.000000	1.000000
75%	117.000000	26.000000	2.000000
max	910.000000	220.000000	38.000000

In [32]: `# ham
df[df['target'] == 0][['num_characters', 'num_words', 'num_sentences']].describe`

```
Out[32]:
```

	num_characters	num_words	num_sentences
count	4516.000000	4516.000000	4516.000000
mean	70.456820	17.123782	1.820195
std	56.356802	13.493970	1.383657
min	2.000000	1.000000	1.000000
25%	34.000000	8.000000	1.000000
50%	52.000000	13.000000	1.000000
75%	90.000000	22.000000	2.000000
max	910.000000	220.000000	38.000000

```
In [33]: #spam
df[df['target'] == 1][['num_characters', 'num_words', 'num_sentences']].describe
```

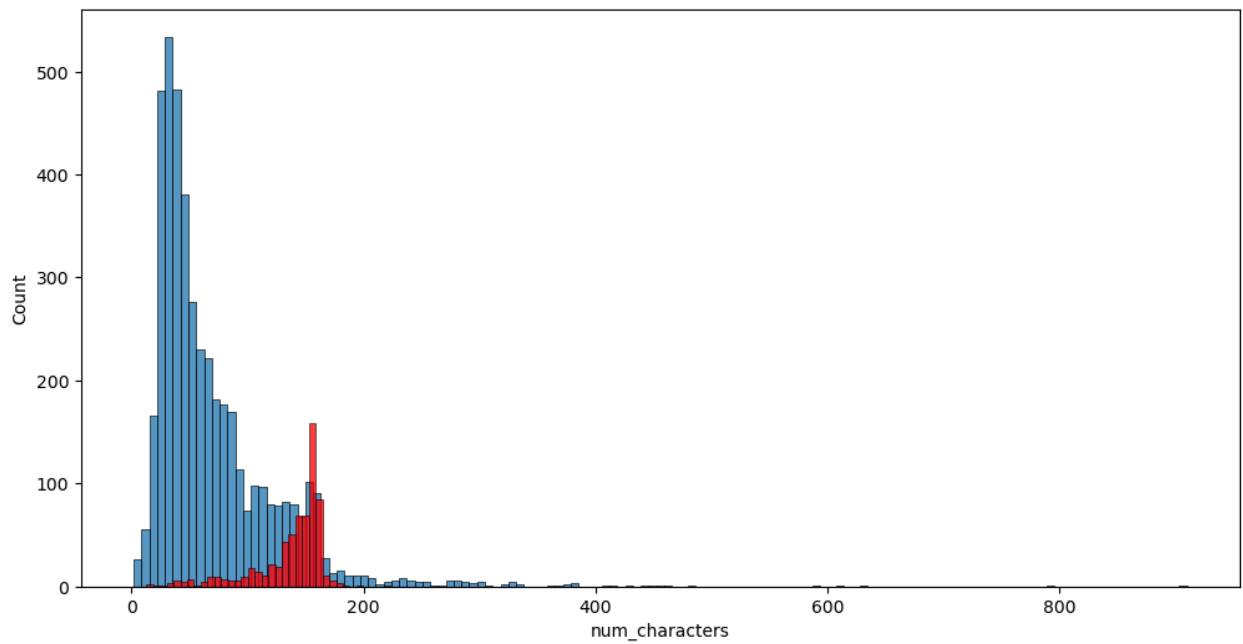
```
Out[33]:
```

	num_characters	num_words	num_sentences
count	653.000000	653.000000	653.000000
mean	137.479326	27.675345	2.978560
std	30.014336	7.011513	1.493185
min	13.000000	2.000000	1.000000
25%	131.000000	25.000000	2.000000
50%	148.000000	29.000000	3.000000
75%	157.000000	32.000000	4.000000
max	223.000000	46.000000	9.000000

```
In [34]: import seaborn as sns
```

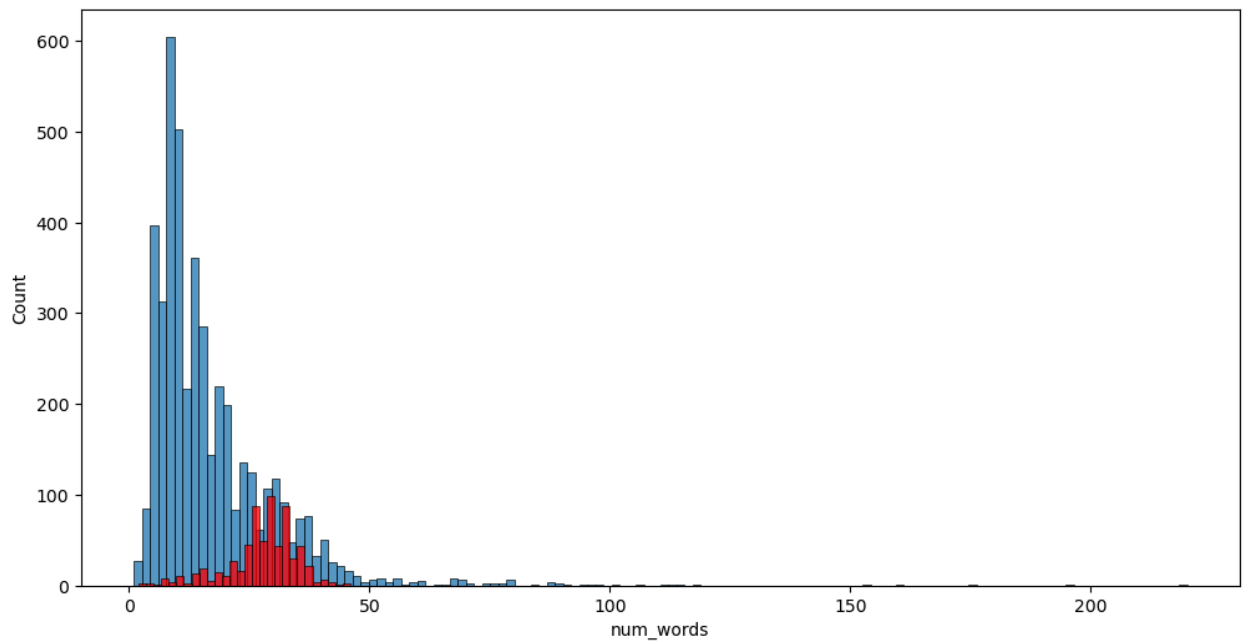
```
In [35]: plt.figure(figsize=(12,6))
sns.histplot(df[df['target'] == 0]['num_characters'])
sns.histplot(df[df['target'] == 1]['num_characters'],color='red')
```

```
Out[35]: <Axes: xlabel='num_characters', ylabel='Count'>
```



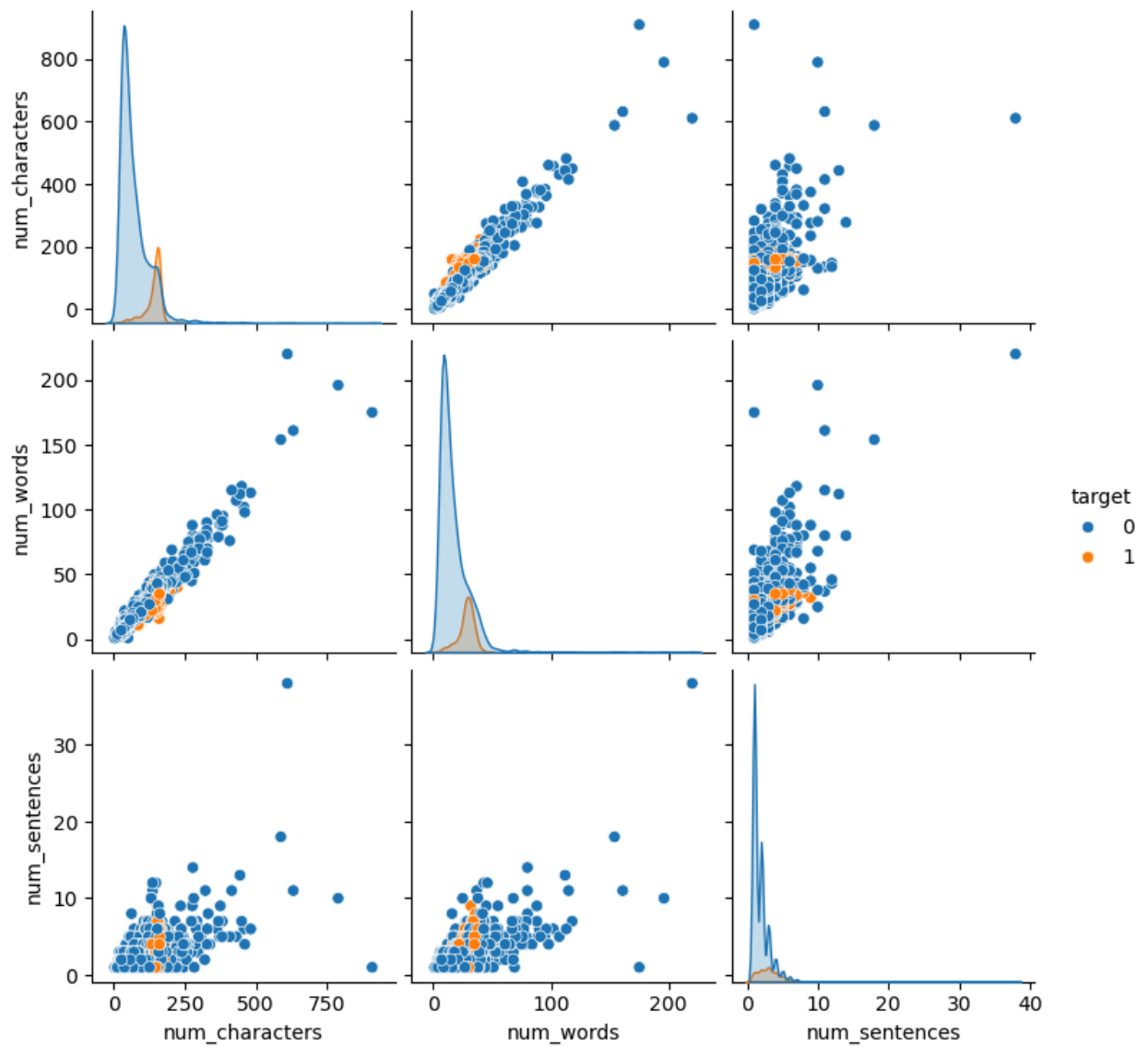
```
In [36]: plt.figure(figsize=(12,6))
sns.histplot(df[df['target'] == 0]['num_words'])
sns.histplot(df[df['target'] == 1]['num_words'],color='red')
```

Out[36]: <Axes: xlabel='num_words', ylabel='Count'>



```
In [37]: sns.pairplot(df,hue='target')
```

Out[37]: <seaborn.axisgrid.PairGrid at 0x1d4804738c0>



```
In [41]: sns.heatmap(df.select_dtypes(include='number').corr(), annot=True, cmap='coolw
plt.show()
```



```

y.clear()

for i in text:
    if i not in stopwords.words('english') and i not in string.punctuation:
        y.append(i)

text = y[:]
y.clear()

for i in text:
    y.append(ps.stem(i))

return " ".join(y)

```

```

[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\monde\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\monde\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!

```

In [49]: `transform_text("I'm gonna be home soon and i don't want to talk about this stu`

Out[49]: 'gon na home soon want talk stuff anymor tonight k cri enough today'

In [50]: `df['text'][10]`

Out[50]: "I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today."

In [51]: `from nltk.stem.porter import PorterStemmer`
`ps = PorterStemmer()`
`ps.stem('loving')`

Out[51]: 'love'

In [52]: `df['transformed_text'] = df['text'].apply(transform_text)`

In [53]: `df.head()`

Out[53]:	target	text	num_characters	num_words	num_sentences	transformed_t
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong p crazi avail bu great wor
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 comp win fa final tkt 2
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli u c already
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	nah think goe live around tho

```
In [56]: !pip install wordcloud
from wordcloud import WordCloud
wc = WordCloud(width=500, height=500, min_font_size=10, background_color='white')
```

Collecting wordcloud

Downloading wordcloud-1.9.4-cp313-cp313-win_amd64.whl.metadata (3.5 kB)

Requirement already satisfied: numpy>=1.6.1 in c:\users\monde\appdata\roaming\python\python313\site-packages (from wordcloud) (2.3.2)

Requirement already satisfied: pillow in c:\users\monde\anaconda3\lib\site-packages (from wordcloud) (11.1.0)

Requirement already satisfied: matplotlib in c:\users\monde\anaconda3\lib\site-packages (from wordcloud) (3.10.0)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\monde\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.3.1)

Requirement already satisfied: cycler>=0.10 in c:\users\monde\anaconda3\lib\site-packages (from matplotlib->wordcloud) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\monde\anaconda3\lib\site-packages (from matplotlib->wordcloud) (4.55.3)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\monde\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.4.8)

Requirement already satisfied: packaging>=20.0 in c:\users\monde\appdata\roaming\python\python313\site-packages (from matplotlib->wordcloud) (25.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\monde\anaconda3\lib\site-packages (from matplotlib->wordcloud) (3.2.0)

Requirement already satisfied: python-dateutil>=2.7 in c:\users\monde\appdata\roaming\python\python313\site-packages (from matplotlib->wordcloud) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in c:\users\monde\appdata\roaming\python\python313\site-packages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.17.0)

Downloading wordcloud-1.9.4-cp313-cp313-win_amd64.whl (300 kB)

Installing collected packages: wordcloud

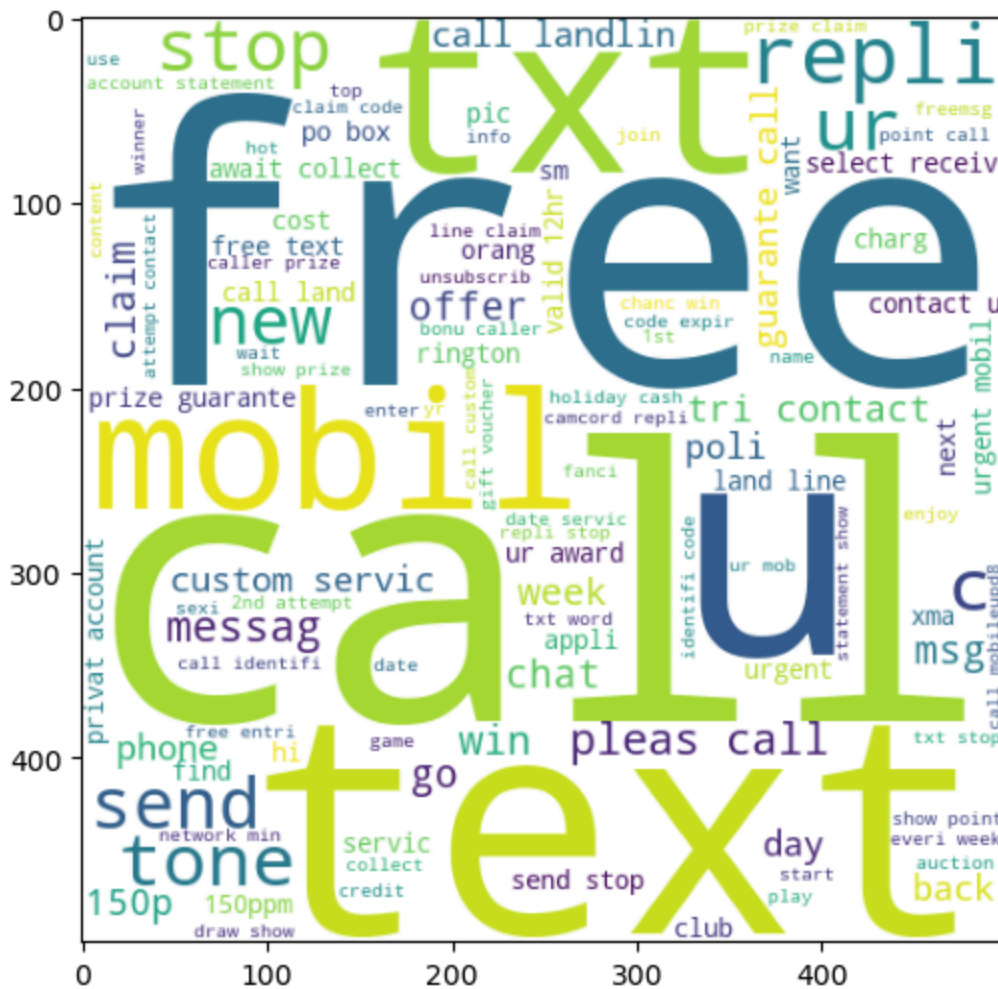
Successfully installed wordcloud-1.9.4

```
In [57]: from wordcloud import WordCloud
wc = WordCloud(width=500, height=500, min_font_size=10, background_color='white')
```

```
In [58]: spam_wc = wc.generate(df[df['target'] == 1]['transformed_text'].str.cat(sep=""))
```

```
In [59]: plt.figure(figsize=(15,6))
plt.imshow(spam_wc)
```

```
Out[59]: <matplotlib.image.AxesImage at 0x1d484fb6660>
```



```
In [60]: ham_wc = wc.generate(df[df['target'] == 0]['transformed_text'].str.cat(sep=" "
```

```
In [61]: plt.figure(figsize=(15,6))
plt.imshow(ham wc)
```

```
Out[61]: <matplotlib.image.AxesImage at 0x1d488d1ac10>
```



```
In [62]: df.head()
```

Out[62]:	target	text	num_characters	num_words	num_sentences	transformed_t
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong p crazi avail bu great wor
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 comp win fa final tkt 2
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli u c already
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	nah think goe live around tho

```
In [63]: spam_corpus = []
for msg in df[df['target'] == 1]['transformed_text'].tolist():
    for word in msg.split():
        spam_corpus.append(word)
```

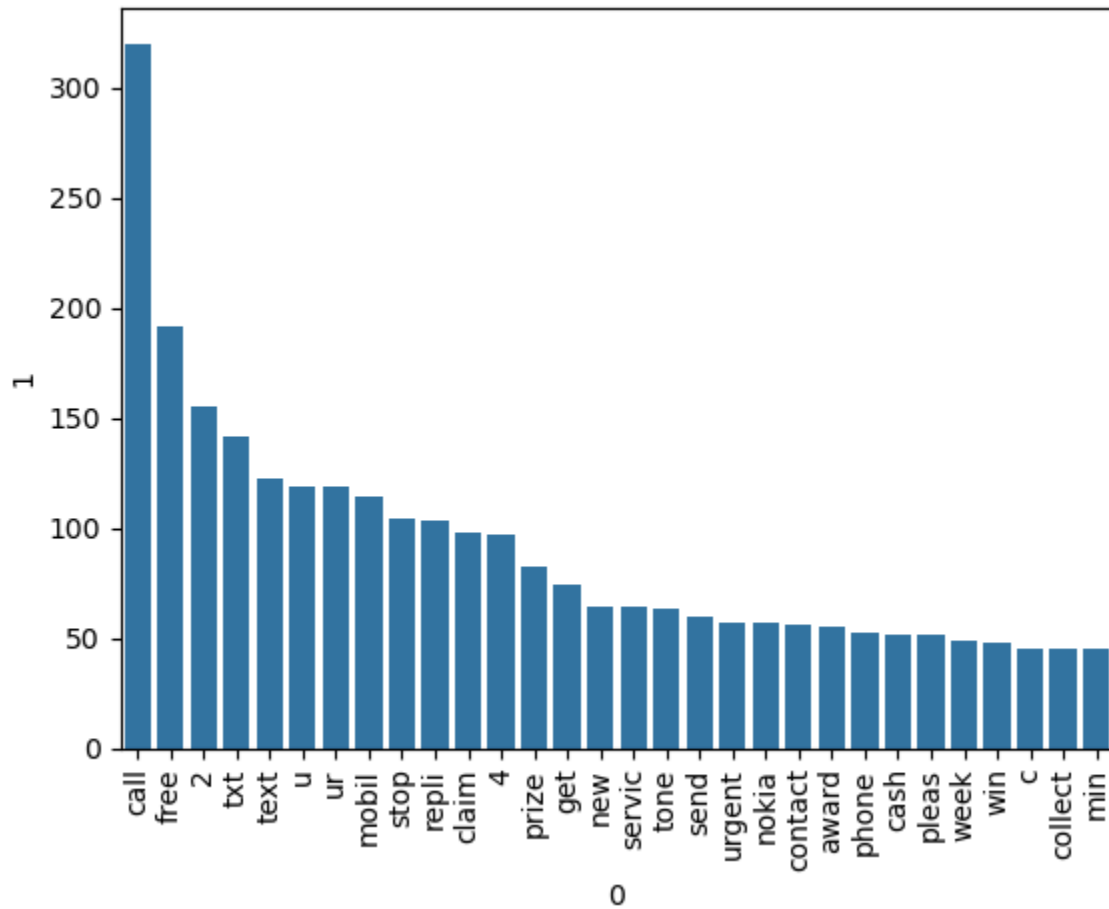
```
In [64]: len(spam_corpus)
```

Out[64]: 9941

```
In [66]: from collections import Counter
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Fixed barplot syntax using x= and y=
sns.barplot(
    x=pd.DataFrame(Counter(spam_corpus).most_common(30))[0],
    y=pd.DataFrame(Counter(spam_corpus).most_common(30))[1])
```

```
)
plt.xticks(rotation='vertical')
plt.show()
```



```
In [67]: ham_corpus = []
for msg in df[df['target'] == 0]['transformed_text'].tolist():
    for word in msg.split():
        ham_corpus.append(word)
```

```
In [68]: len(ham_corpus)
```

```
Out[68]: 35305
```

```
In [69]: # Text Vectorization
# using Bag of Words
df.head()
```

Out[69]:

	target	text	num_characters	num_words	num_sentences	transformed_t
--	--------	------	----------------	-----------	---------------	---------------

0	0	Go until jurong point, crazy.. Available only in ...	111	24	2	go jurong p crazi avail bu great wor
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 comp win fa final tkt 2
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli u c already
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	nah think goe live around tho

In []: *#4.MODEL BUILDING*

```
In [70]: from sklearn.feature_extraction.text import CountVectorizer,TfidfVectorizer
cv = CountVectorizer()
tfidf = TfidfVectorizer(max_features=3000)
```

```
In [71]: X = tfidf.fit_transform(df['transformed_text']).toarray()
```

```
In [ ]: #from sklearn.preprocessing import MinMaxScaler
#scaler = MinMaxScaler()
#X = scaler.fit_transform(X)
```

```
In [ ]: # appending the num_character col to X
#X = np.hstack((X,df['num_characters'].values.reshape(-1,1)))
```

```
In [72]: X.shape
```

Out[72]: (5169, 3000)

```
In [73]: y = df['target'].values
```

```
In [74]: from sklearn.model_selection import train_test_split
```

```
In [75]: X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_stat
```

```
In [76]: from sklearn.naive_bayes import GaussianNB,MultinomialNB,BernoulliNB
from sklearn.metrics import accuracy_score,confusion_matrix,precision_score
```

```
In [77]: gnb = GaussianNB()
mnb = MultinomialNB()
bnb = BernoulliNB()
```

```
In [78]: gnb.fit(X_train,y_train)
y_pred1 = gnb.predict(X_test)
print(accuracy_score(y_test,y_pred1))
print(confusion_matrix(y_test,y_pred1))
print(precision_score(y_test,y_pred1))
```

0.8713733075435203

[[788 108]

[25 113]]

0.5113122171945701

```
In [79]: mnb.fit(X_train,y_train)
y_pred2 = mnb.predict(X_test)
print(accuracy_score(y_test,y_pred2))
print(confusion_matrix(y_test,y_pred2))
print(precision_score(y_test,y_pred2))
```

0.971953578336557

[[896 0]

[29 109]]

1.0

```
In [80]: bnb.fit(X_train,y_train)
y_pred3 = bnb.predict(X_test)
print(accuracy_score(y_test,y_pred3))
print(confusion_matrix(y_test,y_pred3))
print(precision_score(y_test,y_pred3))
```

0.9835589941972921

[[895 1]

[16 122]]

0.991869918699187

```
In [ ]: # tfidf --> MNB
```

```
In [83]: pip install xgboost
```

Collecting xgboost

Downloading xgboost-3.1.1-py3-none-win_amd64.whl.metadata (2.1 kB)

Requirement already satisfied: numpy in c:\users\monde\appdata\roaming\python\python313\site-packages (from xgboost) (2.3.2)

Requirement already satisfied: scipy in c:\users\monde\anaconda3\lib\site-packages (from xgboost) (1.15.3)

Downloading xgboost-3.1.1-py3-none-win_amd64.whl (72.0 MB)

```
----- 0.0/72.0 MB ? eta -:--:--
- ----- 1.8/72.0 MB 11.9 MB/s eta 0:00:06
- ----- 2.4/72.0 MB 8.4 MB/s eta 0:00:09
- ----- 2.9/72.0 MB 4.8 MB/s eta 0:00:15
- ----- 3.4/72.0 MB 3.9 MB/s eta 0:00:18
-- ----- 4.2/72.0 MB 3.9 MB/s eta 0:00:18
-- ----- 4.5/72.0 MB 3.5 MB/s eta 0:00:20
-- ----- 4.7/72.0 MB 3.1 MB/s eta 0:00:22
-- ----- 4.7/72.0 MB 3.1 MB/s eta 0:00:22
-- ----- 5.2/72.0 MB 2.7 MB/s eta 0:00:25
-- ----- 5.5/72.0 MB 2.7 MB/s eta 0:00:25
--- ----- 6.0/72.0 MB 2.6 MB/s eta 0:00:26
--- ----- 6.6/72.0 MB 2.6 MB/s eta 0:00:26
--- ----- 7.1/72.0 MB 2.6 MB/s eta 0:00:26
---- ----- 7.9/72.0 MB 2.6 MB/s eta 0:00:25
---- ----- 8.4/72.0 MB 2.7 MB/s eta 0:00:24
---- ----- 9.2/72.0 MB 2.7 MB/s eta 0:00:24
---- ----- 10.2/72.0 MB 2.8 MB/s eta 0:00:22
---- ----- 11.3/72.0 MB 2.9 MB/s eta 0:00:21
---- ----- 12.1/72.0 MB 3.0 MB/s eta 0:00:20
---- ----- 13.4/72.0 MB 3.2 MB/s eta 0:00:19
---- ----- 14.2/72.0 MB 3.2 MB/s eta 0:00:19
---- ----- 14.7/72.0 MB 3.2 MB/s eta 0:00:19
---- ----- 15.2/72.0 MB 3.1 MB/s eta 0:00:19
---- ----- 15.5/72.0 MB 3.1 MB/s eta 0:00:19
---- ----- 16.0/72.0 MB 3.1 MB/s eta 0:00:19
---- ----- 16.8/72.0 MB 3.0 MB/s eta 0:00:19
---- ----- 17.3/72.0 MB 3.0 MB/s eta 0:00:18
---- ----- 18.1/72.0 MB 3.1 MB/s eta 0:00:18
---- ----- 18.9/72.0 MB 3.1 MB/s eta 0:00:18
---- ----- 19.9/72.0 MB 3.1 MB/s eta 0:00:17
---- ----- 20.7/72.0 MB 3.2 MB/s eta 0:00:17
---- ----- 21.8/72.0 MB 3.2 MB/s eta 0:00:16
---- ----- 22.5/72.0 MB 3.3 MB/s eta 0:00:16
---- ----- 22.8/72.0 MB 3.2 MB/s eta 0:00:16
---- ----- 23.1/72.0 MB 3.2 MB/s eta 0:00:16
---- ----- 23.6/72.0 MB 3.1 MB/s eta 0:00:16
---- ----- 24.1/72.0 MB 3.1 MB/s eta 0:00:16
---- ----- 24.6/72.0 MB 3.1 MB/s eta 0:00:16
---- ----- 25.2/72.0 MB 3.1 MB/s eta 0:00:16
---- ----- 26.0/72.0 MB 3.1 MB/s eta 0:00:15
---- ----- 26.7/72.0 MB 3.1 MB/s eta 0:00:15
---- ----- 27.5/72.0 MB 3.1 MB/s eta 0:00:15
---- ----- 28.6/72.0 MB 3.1 MB/s eta 0:00:14
---- ----- 29.6/72.0 MB 3.2 MB/s eta 0:00:14
---- ----- 30.7/72.0 MB 3.2 MB/s eta 0:00:13
---- ----- 31.7/72.0 MB 3.3 MB/s eta 0:00:13
```

-----	32.2/72.0	MB	3.3	MB/s	eta	0:00:13
-----	32.5/72.0	MB	3.2	MB/s	eta	0:00:13
-----	33.0/72.0	MB	3.2	MB/s	eta	0:00:13
-----	33.6/72.0	MB	3.2	MB/s	eta	0:00:13
-----	34.1/72.0	MB	3.2	MB/s	eta	0:00:12
-----	34.9/72.0	MB	3.2	MB/s	eta	0:00:12
-----	35.7/72.0	MB	3.2	MB/s	eta	0:00:12
-----	36.4/72.0	MB	3.2	MB/s	eta	0:00:12
-----	37.5/72.0	MB	3.2	MB/s	eta	0:00:11
-----	38.5/72.0	MB	3.3	MB/s	eta	0:00:11
-----	39.6/72.0	MB	3.3	MB/s	eta	0:00:10
-----	40.6/72.0	MB	3.3	MB/s	eta	0:00:10
-----	41.4/72.0	MB	3.3	MB/s	eta	0:00:10
-----	41.9/72.0	MB	3.3	MB/s	eta	0:00:10
-----	42.5/72.0	MB	3.3	MB/s	eta	0:00:09
-----	42.7/72.0	MB	3.3	MB/s	eta	0:00:09
-----	42.7/72.0	MB	3.3	MB/s	eta	0:00:09
-----	43.0/72.0	MB	3.2	MB/s	eta	0:00:10
-----	43.3/72.0	MB	3.2	MB/s	eta	0:00:10
-----	43.8/72.0	MB	3.1	MB/s	eta	0:00:09
-----	44.0/72.0	MB	3.1	MB/s	eta	0:00:09
-----	44.6/72.0	MB	3.1	MB/s	eta	0:00:09
-----	45.1/72.0	MB	3.1	MB/s	eta	0:00:09
-----	45.6/72.0	MB	3.1	MB/s	eta	0:00:09
-----	46.4/72.0	MB	3.1	MB/s	eta	0:00:09
-----	47.2/72.0	MB	3.1	MB/s	eta	0:00:08
-----	48.0/72.0	MB	3.1	MB/s	eta	0:00:08
-----	48.8/72.0	MB	3.1	MB/s	eta	0:00:08
-----	49.8/72.0	MB	3.1	MB/s	eta	0:00:08
-----	50.9/72.0	MB	3.2	MB/s	eta	0:00:07
-----	51.9/72.0	MB	3.2	MB/s	eta	0:00:07
-----	52.7/72.0	MB	3.2	MB/s	eta	0:00:07
-----	53.2/72.0	MB	3.2	MB/s	eta	0:00:06
-----	53.7/72.0	MB	3.2	MB/s	eta	0:00:06
-----	54.5/72.0	MB	3.2	MB/s	eta	0:00:06
-----	55.3/72.0	MB	3.2	MB/s	eta	0:00:06
-----	56.1/72.0	MB	3.2	MB/s	eta	0:00:05
-----	57.1/72.0	MB	3.2	MB/s	eta	0:00:05
-----	58.2/72.0	MB	3.2	MB/s	eta	0:00:05
-----	58.7/72.0	MB	3.3	MB/s	eta	0:00:05
-----	59.2/72.0	MB	3.2	MB/s	eta	0:00:04
-----	60.0/72.0	MB	3.2	MB/s	eta	0:00:04
-----	60.6/72.0	MB	3.2	MB/s	eta	0:00:04
-----	61.3/72.0	MB	3.2	MB/s	eta	0:00:04
-----	62.1/72.0	MB	3.2	MB/s	eta	0:00:04
-----	62.7/72.0	MB	3.2	MB/s	eta	0:00:03
-----	63.2/72.0	MB	3.2	MB/s	eta	0:00:03
-----	63.7/72.0	MB	3.2	MB/s	eta	0:00:03
-----	64.2/72.0	MB	3.2	MB/s	eta	0:00:03
-----	64.7/72.0	MB	3.2	MB/s	eta	0:00:03
-----	65.5/72.0	MB	3.2	MB/s	eta	0:00:03
-----	66.3/72.0	MB	3.2	MB/s	eta	0:00:02
-----	67.1/72.0	MB	3.2	MB/s	eta	0:00:02
-----	67.9/72.0	MB	3.2	MB/s	eta	0:00:02

```

----- - 68.9/72.0 MB 3.2 MB/s eta 0:00:01
----- - 70.0/72.0 MB 3.3 MB/s eta 0:00:01
----- 71.0/72.0 MB 3.3 MB/s eta 0:00:01
----- 71.8/72.0 MB 3.3 MB/s eta 0:00:01
----- 72.0/72.0 MB 3.3 MB/s eta 0:00:00

```

Installing collected packages: xgboost

Successfully installed xgboost-3.1.1

Note: you may need to restart the kernel to use updated packages.

In [84]: `!pip install xgboost`

Requirement already satisfied: xgboost in c:\users\monde\anaconda3\lib\site-packages (3.1.1)

Requirement already satisfied: numpy in c:\users\monde\appdata\roaming\python\python313\site-packages (from xgboost) (2.3.2)

Requirement already satisfied: scipy in c:\users\monde\anaconda3\lib\site-packages (from xgboost) (1.15.3)

In [85]: `from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier`

In [86]: `svc = SVC(kernel='sigmoid', gamma=1.0)
knc = KNeighborsClassifier()
mnb = MultinomialNB()
dtc = DecisionTreeClassifier(max_depth=5)
lrc = LogisticRegression(solver='liblinear', penalty='l1')
rfc = RandomForestClassifier(n_estimators=50, random_state=2)
abc = AdaBoostClassifier(n_estimators=50, random_state=2)
bc = BaggingClassifier(n_estimators=50, random_state=2)
etc = ExtraTreesClassifier(n_estimators=50, random_state=2)
gbdt = GradientBoostingClassifier(n_estimators=50, random_state=2)
xgb = XGBClassifier(n_estimators=50, random_state=2)`

In [87]: `clfs = {
 'SVC' : svc,
 'KN' : knc,
 'NB' : mnb,
 'DT' : dtc,
 'LR' : lrc,
 'RF' : rfc,
 'AdaBoost' : abc,
 'BgC' : bc,
 'ETC' : etc,
 'GBDT' : gbdt,`

```
'xgb':xgb  
}
```

```
In [88]: def train_classifier(clf,X_train,y_train,X_test,y_test):  
        clf.fit(X_train,y_train)  
        y_pred = clf.predict(X_test)  
        accuracy = accuracy_score(y_test,y_pred)  
        precision = precision_score(y_test,y_pred)  
  
        return accuracy,precision
```

```
In [89]: train_classifier(svc,X_train,y_train,X_test,y_test)
```

```
Out[89]: (0.9758220502901354, 0.9747899159663865)
```

```
In [90]: accuracy_scores = []  
        precision_scores = []  
  
        for name,clf in clfs.items():  
  
            current_accuracy,current_precision = train_classifier(clf, X_train,y_train  
  
            print("For ",name)  
            print("Accuracy - ",current_accuracy)  
            print("Precision - ",current_precision)  
  
            accuracy_scores.append(current_accuracy)  
            precision_scores.append(current_precision)
```

```
For SVC
Accuracy - 0.9758220502901354
Precision - 0.9747899159663865
For KN
Accuracy - 0.9052224371373307
Precision - 1.0
For NB
Accuracy - 0.971953578336557
Precision - 1.0
For DT
Accuracy - 0.9323017408123792
Precision - 0.8333333333333334
For LR
Accuracy - 0.9564796905222437
Precision - 0.9696969696969697
For RF
Accuracy - 0.971953578336557
Precision - 0.9658119658119658
For AdaBoost
Accuracy - 0.9216634429400387
Precision - 0.8202247191011236
For BgC
Accuracy - 0.9593810444874274
Precision - 0.8636363636363636
For ETC
Accuracy - 0.9787234042553191
Precision - 0.9754098360655737
For GBDT
Accuracy - 0.9497098646034816
Precision - 0.9215686274509803
For xgb
Accuracy - 0.9700193423597679
Precision - 0.9572649572649573
```

```
In [93]: performance_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy':accuracy_scc
```

```
In [94]: performance_df
```

Out[94]:

	Algorithm	Accuracy	Precision
1	KN	0.905222	1.000000
2	NB	0.971954	1.000000
8	ETC	0.978723	0.975410
0	SVC	0.975822	0.974790
4	LR	0.956480	0.969697
5	RF	0.971954	0.965812
10	xgb	0.970019	0.957265
9	GBDT	0.949710	0.921569
7	BgC	0.959381	0.863636
3	DT	0.932302	0.833333
6	AdaBoost	0.921663	0.820225

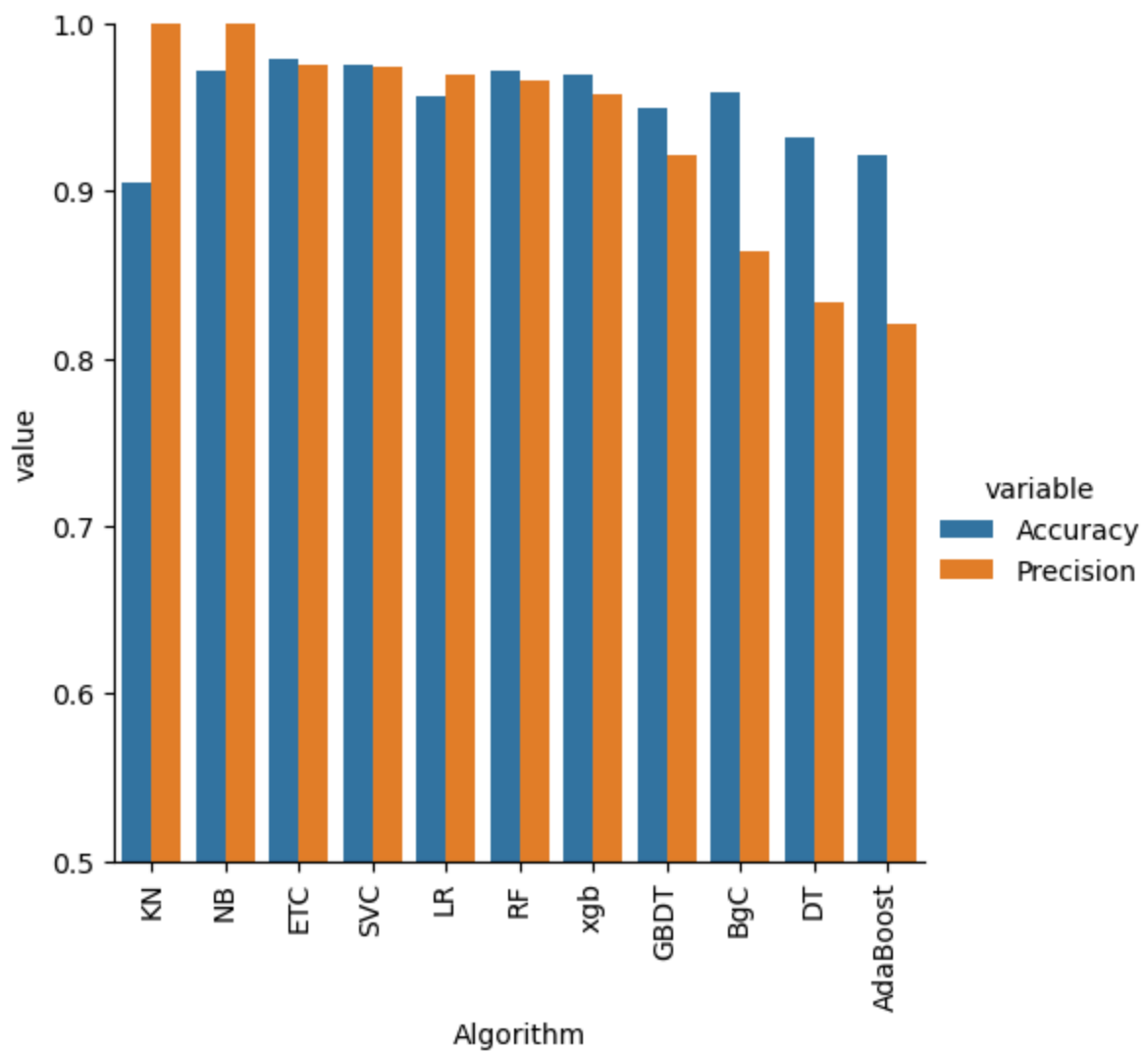
```
In [95]: performance_df1 = pd.melt(performance_df, id_vars = "Algorithm")
```

```
In [96]: performance_df1
```

Out[96]:

	Algorithm	variable	value
0	KN	Accuracy	0.905222
1	NB	Accuracy	0.971954
2	ETC	Accuracy	0.978723
3	SVC	Accuracy	0.975822
4	LR	Accuracy	0.956480
5	RF	Accuracy	0.971954
6	xgb	Accuracy	0.970019
7	GBDT	Accuracy	0.949710
8	BgC	Accuracy	0.959381
9	DT	Accuracy	0.932302
10	AdaBoost	Accuracy	0.921663
11	KN	Precision	1.000000
12	NB	Precision	1.000000
13	ETC	Precision	0.975410
14	SVC	Precision	0.974790
15	LR	Precision	0.969697
16	RF	Precision	0.965812
17	xgb	Precision	0.957265
18	GBDT	Precision	0.921569
19	BgC	Precision	0.863636
20	DT	Precision	0.833333
21	AdaBoost	Precision	0.820225

```
In [97]: sns.catplot(x = 'Algorithm', y='value',  
                    hue = 'variable',data=performance_df1, kind='bar',height=5)  
plt.ylim(0.5,1.0)  
plt.xticks(rotation='vertical')  
plt.show()
```



```
In [ ]: # model improve
        # 1. Change the max_features parameter of TfIdf
```

```
In [98]: emp_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy_max_ft_3000':accuracy
```

```
In [99]: temp_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy_scaling':accuracy_sc
```

```
In [100... new_df = performance_df.merge(temp_df,on='Algorithm')
```

```
In [101... new_df_scaled = new_df.merge(temp_df,on='Algorithm')
```

```
In [102... temp_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy_num_chars':accuracy_
```

```
In [103... new_df_scaled.merge(temp_df,on='Algorithm')
```

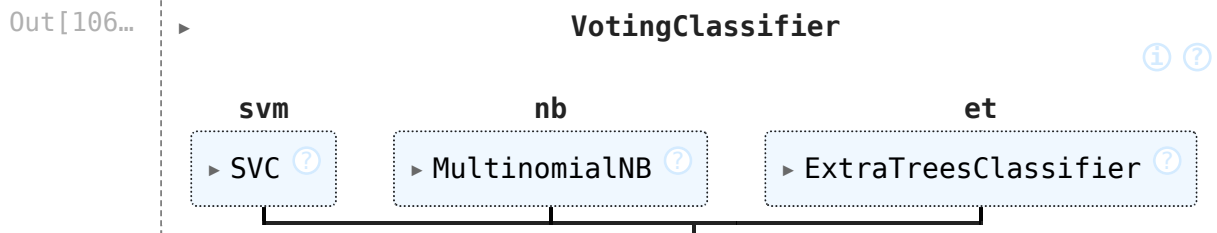
Out[103...	Algorithm	Accuracy	Precision	Accuracy_scaling_x	Precision_scaling_x	Acc
0	KN	0.905222	1.000000	0.905222	1.000000	
1	NB	0.971954	1.000000	0.971954	1.000000	
2	ETC	0.978723	0.975410	0.978723	0.975410	
3	SVC	0.975822	0.974790	0.975822	0.974790	
4	LR	0.956480	0.969697	0.956480	0.969697	
5	RF	0.971954	0.965812	0.971954	0.965812	
6	xgb	0.970019	0.957265	0.970019	0.957265	
7	GBDT	0.949710	0.921569	0.949710	0.921569	
8	BgC	0.959381	0.863636	0.959381	0.863636	
9	DT	0.932302	0.833333	0.932302	0.833333	
10	AdaBoost	0.921663	0.820225	0.921663	0.820225	

```
In [104... # Voting Classifier
svc = SVC(kernel='sigmoid', gamma=1.0, probability=True)
mnb = MultinomialNB()
etc = ExtraTreesClassifier(n_estimators=50, random_state=2)

from sklearn.ensemble import VotingClassifier
```

```
In [105... voting = VotingClassifier(estimators=[('svm', svc), ('nb', mnb), ('et', etc)],
```

```
In [106... voting.fit(X_train, y_train)
```



```
In [107... y_pred = voting.predict(X_test)
print("Accuracy", accuracy_score(y_test, y_pred))
print("Precision", precision_score(y_test, y_pred))
```

Accuracy 0.9796905222437138
Precision 0.9834710743801653

```
In [108... # Applying stacking
estimators=[('svm', svc), ('nb', mnb), ('et', etc)]
final_estimator=RandomForestClassifier()
```

```
In [109... from sklearn.ensemble import StackingClassifier
```

```
In [110... clf = StackingClassifier(estimators=estimators, final_estimator=final_estimator)
```

```
In [111... clf.fit(X_train,y_train)
y_pred = clf.predict(X_test)
print("Accuracy",accuracy_score(y_test,y_pred))
print("Precision",precision_score(y_test,y_pred))
```

Accuracy 0.9806576402321083
Precision 0.9538461538461539

```
In [112... import pickle
pickle.dump(tfidf,open('vectorizer.pkl','wb'))
pickle.dump(mnb,open('model.pkl','wb'))
```

```
In [ ]:
```